

OneCode：我们如何用离散数学 构建一个安全、完全可控的 Agent 内核

博客写作设计稿

大模型可以生成内容、计划和工具调用建议，但不应拥有最终执行权。OneCode 在模型输出与真实行动之间建立确定性内核，将不确定的模型输出转化为有限、受约束、可验证、可追溯、可恢复的执行过程。

一、写作目标

这是一篇介绍 OneCode 的中文技术博客，主要面向 Agent 开发者和技術负责人，同时让更广泛的 AI 从业者也能读懂。

文章不是通用 Agent 教程、产品说明书或技术白皮书，而是一篇关于 OneCode 的技术宣言：清楚说明我们做了什么、为什么这样做、具备哪些优势，同时不披露构成技术壁垒的核心公式、参数和决策规则。

二、核心价值

安全

可控

可验证

可恢复

自主知识产权、独立实现、模型无关、本地优先、证据防篡改和工程验证，是支撑这四个核心价值的项目优势。

三、可公开的数学表达

文章只使用解释设计思想所必需的抽象公式，不与真实状态位、映射关系、转移条件或内部决策表建立对应。

$$X = \{0, 1\}^n$$

有限离散状态空间

$$\phi : E \rightarrow X$$

将输入、环境和运行结果等证据投影为受控状态

$$T : X \times I \rightarrow X$$

状态根据有效输入进行确定性转移

$$D : X \rightarrow A$$

内核只能从有限、受约束的行动集合中作出决定

全文只使用现代工程和离散数学语言，不讨论传统文化、玄学或内部规则体系的历史来源，统一表述为“OneCode 自主研发的形式化规则体系”。

四、正文结构

1. 为什么我们要做 OneCode

模型生成能力不等于安全可靠的执行能力。OneCode 的目标不是最大化模型的自由，而是控制模型是否可以行动，以及行动如何影响真实环境。

2. OneCode 是什么

OneCode 是我们从底层独立设计和实现的安全、完全可控的 Agent 内核，拥有自主知识产权和完整的技术演进控制权。它不是现有 Agent 框架的二次封装。

3. 安全与可控如何实现

模型输出、Prompt、Skill、项目规则和外部建议都只是候选信息或证据，不拥有执行权。未知、异常或相互冲突的证据不会获得猜测性授权。

4. 一次任务如何通过内核

用“输入 -> 证据 -> 状态 -> 安全裁决 -> 受控行动 -> 验证 -> 证据闭环 -> 交付”展示完整链路。

5. OneCode 的核心优势

集中说明安全内生、确定性控制、独立验证完成、证据防篡改、确定性恢复、可重放、可审计、模型无关、本地优先和独立实现。

6. 我们真正构建的是什么

OneCode 不是聊天界面，也不是工具调用包装层。它是位于模型与真实行动之间的可信执行内核。

五、任务闭环与完成条件

输入 -> 证据 -> 状态 -> 安全裁决 -> 受控行动
验证 -> 证据闭环 -> 交付

模型不能自行宣布任务完成。任务进入可交付状态，必须同时满足以下条件：

- 状态有效
- 物理结果符合约束
- 验证通过
- 关键证据完整

六、OneCode 核心优势

- 安全内生与默认拒绝

- 确定性、有限且受约束的执行权
- 模型不能自行证明任务完成
- 追加式、可交叉验证的防篡改证据
- 确定性恢复、幂等重试和冲突识别
- 可重现、可重放、可追溯、可审计
- 模型无关与本地数据控制
- 轻量核心，不依赖现成 Agent 框架
- 单一权威链，避免组件越权和策略漂移
- 自主设计、自主实现和自主技术演进
- 完整的自动化验证与数学审计能力

七、保密边界

可以公开

- 有限状态和确定性控制的抽象思想
- Agent 任务的外部生命周期
- 安全、证据、验证和恢复原则
- 已验证的项目能力和结果
- 项目的独立实现与自主知识产权

不能公开

- 真实证据到状态的映射
- 内部状态位含义
- 状态转移表和决策矩阵
- 权重、阈值、优先级和平衡参数
- 冲突消解、恢复、调制和调度规则
- 足以复刻内核的伪代码与实现细节

八、表述与准确性边界

- 使用清晰、克制、简洁的中文，少用术语，不做冗长推导，不为篇幅补充内容。
- 将 OneCode 表述为确定性、本地优先的 Agent 内核，不表述为全能型自主助手。
- 客观表述自主知识产权和实现控制权，不把 GPL v3 开源误解为放弃著作权。
- 区分仓库已验证能力与未来部署能力，不把本地确定性基准误述为在线模型的原始幻觉率。
- 没有固定字数。约 2000 字只是自然方向，以完整、准确表达 OneCode 为准。

**OneCode 不是另一个会调用工具的聊天机器人，
而是位于模型与真实行动之间、负责控制执行权的可信 Agent 内核。**